MBARARA UNIVERSITY OF SCIENCE AND TECHNOLOGY.

SOFTWARE ENGINEERING

GROUP DISCUSSION 2.

BRIDGE TECH GROUP.

<u>GROUP NAMES</u>.

ARINDA LETICIA 2024/BCS/043/PS

ARYAMANYA LUCKY 2024/BCS/045/PS

TUMUKUNDE ISAAC 2024/BCS/162/PS

KUSINGURA JACHIN 2024/BCS/091/PS

NANJUKI DAPHINE 2024/BCS/129/PS

NAMBATYA BLADINA DOREEN 2024/BCS/128/PS

PART 1: Understanding Software.

Activity 1: What Is Software — Beyond Code?

Discussion Questions:

Is software just code? Why or why not?

No, software is more than just code. Code is the executable instructions that tell the computer what to do, but software is the entire logical system. It includes data structures that organize information and, crucially, documentation that explains how to use, maintain, and understand the system. Without documentation, code becomes a black box that is difficult to fix or improve.

Why is documentation considered part of software?

Documentation is considered part of software because it is essential for its long-term viability. It supports the software's maintainability (allowing developers to fix bugs) and acceptability (allowing users to understand how to use it). Without documentation, the software cannot be effectively evolved or supported.

Why does software not “wear out” like hardware?

Hardware fails due to physical wear and tear (e.g., dust, heat, component degradation). Software is logical, not physical. It doesn't rust or break from age. However, it can fail because the underlying environment (operating systems, user needs) changes, or because undiscovered defects exist.

Why does software deteriorate over time?

Software deteriorates due to constant change. As bugs are fixed and features are added, the original architecture can become corrupted if changes aren't managed carefully. This is often called "software entropy" or "bit rot." The code becomes increasingly complex and disorganized, making future changes harder and introducing new errors.

Application to Semester Project: Documentation Requirements

Reflection Task:

Examples of software

1. Google Maps
   Program: The navigation algorithms, UI rendering engine, GPS listener.

Data: Map tiles, saved home/work addresses, search history, real-time traffic data.

Documentation: Explicit: Help articles, tutorial videos. Implicit: Intuitive icons (magnifying glass for search), gestures (pinch to zoom).

2. Microsoft Word
   Program: The executable code that handles text rendering, spell check, file I/O.

   Data: The .docx files you create, templates, user settings/preferences.

   Documentation: Explicit: Built-in "Tell Me" assistant, online support, user manuals. Implicit: Ribbon interface layout grouped by function (Home, Insert, etc.).

3. WhatsApp
   Program: The messaging protocol code, encryption logic, UI for chat threads.

   Data: The messaging protocol code, encryption logic, UI for chat threads.

   Documentation: The messaging protocol code, encryption logic, UI for chat threads.

To be considered "complete software," our semester project will require:

1. Requirements Document: To define what we are building and ensure agreement with the client/instructor.
2. Design Diagrams: To map the architecture (UML diagrams, database schema) so future developers can understand the structure.
3. User Guide: To help end-users install and operate the software effectively.
4. Test Cases: To verify the software works correctly and to ensure future updates don't break existing functionality (regression testing).

PART 2: Types of Software and Project Vision

Activity 2**: What Type of Software Will You Build?**

HealthBridge by bridge tech.

Project Vision: Community Health Resource Locator.

A web application that helps people find nearby healthcare services, medicine availability, and health education resources in underserved communities.

Step 1: Categorize Your Project

Web-based system and Data processing system.

Step 2: Implications Discussion

1. **What are the key risks?**

   Data accuracy : Outdated hospital/medicine information

   Scalability : Slow performance with many users

   API dependencies : Map services or external health APIs failing

2. **What quality attributes are most critical?**

   Reliability :Accurate, up-to-date information

   Usability : Simple for elderly or low-tech users

   Security :Protect user location data and reviews

3. **What challenges might you face?**

1. Security: Risk of data breaches (SQL injection, XSS attacks) due to exposure over the internet.
2. Concurrency: Risk of data corruption if multiple users try to edit the same data simultaneously.
3. Availability: Risk of downtime; the system must be reliable and accessible 24/7.

PART 3: Attributes of Good Software

Which attribute is most important for your project?

Dependability is paramount. If the software says "Free Clinic at 123 Main St" and the user walks there to find it closed or non-existent, the software has failed its

purpose. Acceptability (Usability) is second, as the users might be in distress and need a frictionless experience.

Can improving one attribute negatively affect another?

Yes.

Security vs. Usability: Requiring users to create a complex password and verify their email (Security) might stop a homeless person from quickly finding a shelter (Usability).

Efficiency vs. Maintainability: Hard-coding clinic locations directly into the code might make the search very fast (Efficiency), but it makes it impossible for community partners to update their own hours later (Maintainability).

How will you measure quality in your project?

Dependability: Track false reports, how many users report a resource is missing or incorrect.
Acceptability: Measure time to find resource, how quickly a new user can find a dentist.
Efficiency: Measure map load times and search result speed.

Critical Thinking:

If your system works but is difficult to maintain, is it good software?

No. If HealthBridge is hard to maintain, the clinic hours will quickly become outdated. Within 6 months, the software will be directing people to wrong locations, making it not just useless, but harmful to the community.

<u>PART 4: The Software Engineering Process.</u>

Activity 4**: Map Your Project to the Lifecycle**

What stakeholders must you consult?

End Users: Low-income families, elderly individuals.
Resource Providers: Local clinics, food banks, non-profit administrators.
Data Curators: The person responsible for keeping the database accurate.

What risks exist in the specification stage?

Project expansion: Trying to list *every* type of resource instead of focusing on the most critical one's examples primary care vs. pet care.
Missing the User Need: Assuming users want a fancy app, when they might actually prefer an SMS text service if they don't have smartphones.

How will you validate your system?

We will take a prototype to a community center and ask real users to find specific resources. We will watch where they click and ask if the results were helpful.

What changes might occur after delivery?

Clinics close or change hours. New government funding might create new health programs. Phone numbers change.

Lifecycle Roadmap for HealthBridge.

Specification: Interview clinic staff, patients, community leaders
Design: Wireframes, database of facilities/medicines, API design
Development: involves building the system step by step. Example Build search + admin panel
Validation: The system is tested with real users to check if it works as expected.
Evolution: This involves planning future improvements such as planning for adding telemedicine, appointment booking.

<u>PART 5: Modern Challenges (15 minutes)</u>

Activity 5**: Reality Check**
Discuss how these challenges affect your project:
Heterogeneity: Must work on phones, tablets, low-end devices
Security: Protect user health searches and location
Rapid change: Medicine prices/availability change daily
Scale: Could grow from 1 city to nationwide
Time pressure: Community needs it now.

Scenario Challenge:
Situation: You have 4 weeks remaining. The client asks you to add "Pharmacy" listings (major requirement change). Your team discovers a security vulnerability where a user's

search for "HIV clinic" is exposed in the browser history.

Priority Order:

1. **Fix the Security Vulnerability:** Exposing sensitive health searches is a massive ethical and legal violation (HIPAA/GDPR concerns). This is priority #1.
2. **Communicate with the Client:** Explain that the "Pharmacy" feature will delay the project, but it can be added if another feature is cut, or deferred to "Phase 2."
3. **Secure the existing data** before adding new features.
   Rationale: Health data privacy is sacred. You cannot launch a health app that leaks data, regardless of how many features it has.

PART 6: Ethics and Professional Responsibility.

Activity 6**: Ethical Reflection**

Dilemma 1: Known minor bug before submission.

Scenario: A bug shows a clinic's address is wrong, but you know the clinic is still generally in that neighborhood.

Action: Report it. A wrong address in a health app can mean someone misses an appointment or walks miles to the wrong place. It must be fixed or clearly labeled as "estimated location."

Dilemma 2: Teammate copies code without attribution.

Scenario: A teammate copies Google Maps integration code from a tutorial without credit.

Action: Explain that this is plagiarism. While using APIs is fine, passing off tutorial code as original work is dishonest. If the tutorial had a license requiring attribution, we must add it.

Dilemma 3: Client requests a feature that compromises user privacy.

Scenario: A hospital sponsor asks HealthBridge to track which users search for "abortion clinics" and report that data to them.

Action: Refuse. This is an extreme violation of privacy. Reference the ACM/IEEE code: "Software engineers shall act consistently with the public interest." We would lose all trust and potentially face legal action.

PART 7: Software Myths.

Which myth is most dangerous for your project?

Myth: *"Once the program works, the job is done."*

Why? For HealthBridge, the code working today doesn't matter if the data is outdated tomorrow. If we think we are "done" at launch, the resource list will rot, and users will stop trusting the app.

How will you avoid it?

We will build a "Data Maintenance Plan" from day one. This includes building an easy-to-use admin panel so non-technical staff can update clinic hours, and setting calendar reminders to verify data every month.